

workflow in a number of ways. Pre-commit, the validity of the file's syntax is checked, avoiding breaking the developer's build process with broken files. It also allows fine-grained permissions to files the translators need access to. Post-commit, Transifex can notify language leaders and others about file submissions, provide RSS feeds for submissions, etc.

Transifex currently supports git, hg, cvs, svn and bzr, and adding more VCSs is a matter of writing a few lines of code. Its developers serve POT-based projects, and are looking forward to extending the i18n support to include intltool-based projects (GNOME), XLIFF, etc. The login mechanism also supports OpenID.

Development of Transifex

The development of Transifex began as part of the 2007 Google Summer of Code project by myself (Oh! Hi! I'm Dimitris Glezos :-)). It was initially written in Python using the TurboGears framework, and right after the summer it was put into production in Fedora, used by more than 100 projects and 500 translators.

Next year, Transifex was presented in more than 10 international conferences, including FOSS in 2008. In the summer, Transifex earned three more GSoC applications and was re-written from scratch using the Django Python framework, now including many of the suggestions from existing users. Development has taken place since then on transifex.org and on the [transifex-devel](mailto:transifex-devel@lists.fedoraproject.org) mailing list.

In the meantime, other projects liked the platform and joined in our efforts. GNOME's Damned Lies and Vertimus tools migrated their code to Django, with the goal of being merged with Transifex at some point in the future.

Future features

With more contributors joining in the developer team, Transifex is now moving towards a stabilised platform to serve independent and upstream software projects and then on to bigger ones.

One of the immediate features we'd like to add is per-VCS file monitoring, so that translators can 'track' a project and get notified when the translation percentage for their language changes. Adding commenting support for projects and submissions, as well as developing support for file uploads will enable translators to better collaborate in QA.

Another often requested feature is the development of a command-line interface allowing translators to do something like the following:

```
$ tx set-language bn_IN
$ tx get-collection Fedora
Received anaconda/po/bn_IN.po
Received packagekit/po/bn_IN.po
$ # Translation...
```

```
$ tx send-collection Fedora
Sent 'anaconda/po/bn_IN.po' (100% translated)
```

The vision: Transifex.net

As mentioned earlier, Transifex allows downstream communities to send files directly to the VCS of upstream projects. One might wonder then, which Transifex community should an independent project choose to receive translations from—Fedora, GNOME, or example.com?

Having a common place where open source translations take place is key to link translation communities together and reach new levels of collaboration between translation teams. Here's a plan we're evolving with www.transifex.net: Establish a healthy network where developers can translate their applications and translators can contribute to their favourite projects. Project teams that wouldn't like to undertake the trouble of setting up their own Transifex instance, should have a stable, rich-in-features service, to join their efforts with the rest of the open source community, under a common umbrella.

Becoming a contributor

Transifex is written in Python and utilises the awesome Django branch with its infamous top-notch documentation. This makes it really easy for folks to join in and extend the platform with the features they'd like to see added. Development information can be found at transifex.org/wiki/Development. To set up a development environment of your own, check out the documentation at docs.transifex.org/intro/install.html.

An example of an easy task would be to add support for associating registered projects with their maintainers/developers. This will give translators a contact point for more information on the project and for conflict resolution. Creating a patch that adds simple support for project maintainers is a matter of a few lines of code: add a foreign key from the *Project* model to the *User* and probably edit the *User Profile* page to include a section listing the projects the user maintains.

Adding support for more VCSs and i18n back-ends is also quite feasible because of the abstractions Transifex includes in those areas. For most needs, one just copies a Python file and changes accordingly. We've marked quite a few tickets with the 'easy_task' keyword, so check out transifex.org/report/9 to start hacking.

Let a thousand languages bloom!  **END**

By: Dimitris Glezos

The author is a member of the Fedora Board and the current Fedora Localisation Leader. He loves watching people read content in their native language, and so he founded Indifex, a fresh start-up specialising in quality and effective translation services. He lives in Greece and likes hacking, photography and rock climbing.